Nile University (NU) IoT Research Internship Report

Day 5

Overview

Day 5 marked a shift from Raspberry Pi development to embedded systems programming using Texas Instruments' CC1352P7 microcontroller. The day covered the TI SimpleLink SDK, Bluetooth Low Energy (BLE) communication concepts, and wireless control between devices using CC1352P7 LaunchPad kits.

Task 1: Code Composer Studio Setup

Installed Code Composer Studio (CCS), TI's integrated development environment, along with the SimpleLink CC13xx/CC26xx SDK. Configured the workspace and project settings and learned the TI project structure and build system.

This was the foundation step for the day, equivalent in role to setting up the Raspberry Pi on Day 1: no embedded development could begin without a working toolchain for the new hardware platform.

Task 2: SimpleLink SDK Exploration

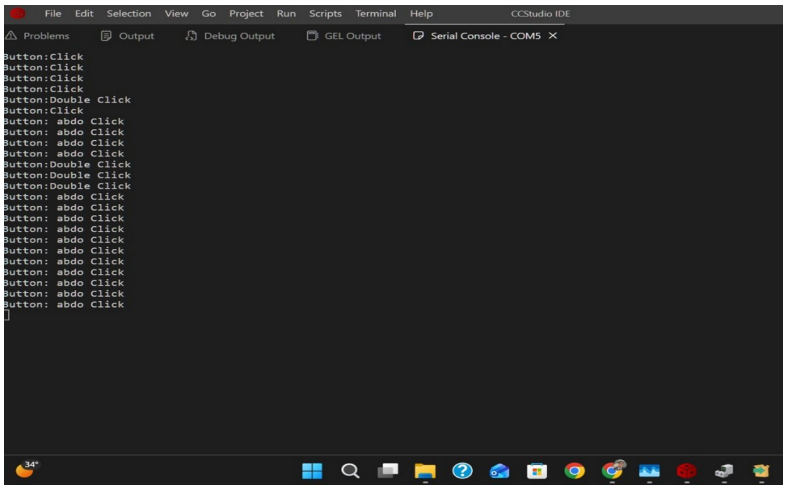
Explored the SimpleLink SDK architecture, learned about the BLE stack implementation, reviewed the structure of example projects (simple_peripheral, simple_central), and understood how TI-RTOS integrates with the SDK.

Improvement over Task 1: Task 1 only got the development environment installed and running. This task built understanding of what the SDK actually provides, which was necessary before any of the later BLE examples could be modified or understood correctly.

Task 3: LED Example - Button Controlled LED

Loaded the LED example onto a single CC1352P7 kit, configured GPIO pins for the LED and button, implemented a button press to toggle the LED, and tested basic GPIO functionality.

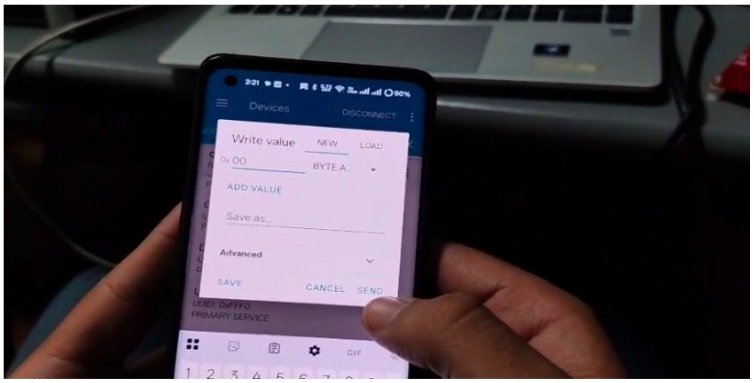
Improvement over Task 2: this task moved from reading SDK documentation and structure to actually running and testing code on the hardware, confirming that the toolchain and board were correctly set up before moving to wireless communication.

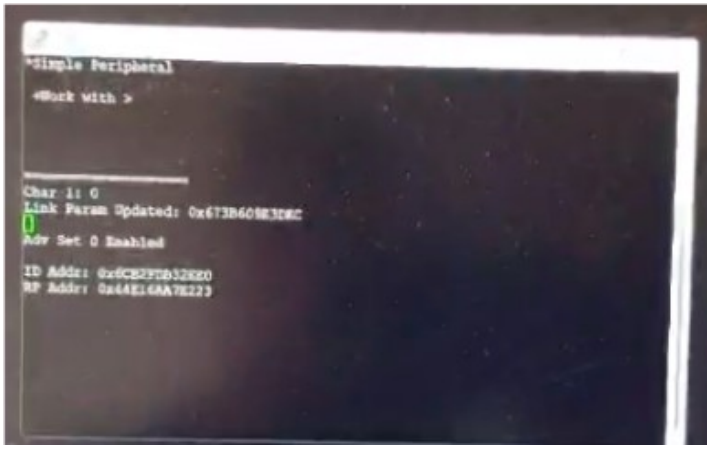


Task 4: BLE Connection with Mobile (nRF Connect)

Installed the nRF Connect app on a smartphone, enabled Bluetooth, and ran the simple_peripheral example on a CC1352P7 kit. The device was discovered and connected to using nRF Connect, allowing the LED to be controlled via BLE characteristics and data to be sent and received between the mobile app and the microcontroller.

Improvement over Task 3: Task 3 controlled the LED using a physically wired button. This task replaced that direct, wired control with wireless BLE control from an external device, introducing the wireless communication concepts (advertising, connecting, characteristics) that the rest of the day built on.



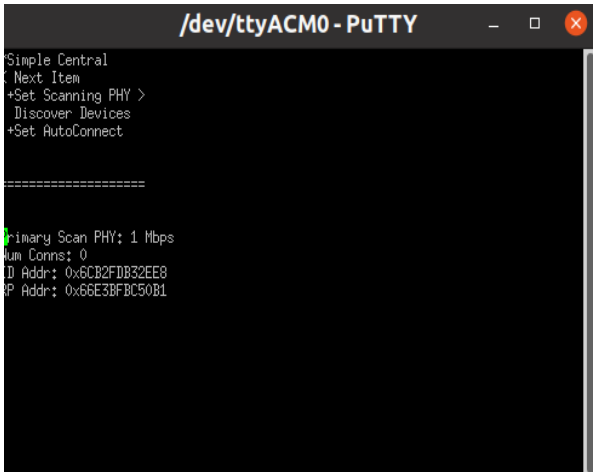


Task 5: BLE Communication Between Two Kits

Flashed simple_peripheral on Kit 1 (peripheral/server) and simple_central on Kit 2 (central/client). Kit 2 scanned for and discovered Kit 1 using its MAC address, established a BLE connection, and the two kits exchanged messages using BLE characteristics.

Improvement over Task 4: Task 4 proved BLE communication worked between a phone and one kit. This task extended that to two microcontrollers communicating directly with each other,

without a phone involved, which is closer to how BLE would be used in an actual embedded device-to-device system.



Debugging with Putty

Configured a UART serial connection and used Putty as a serial terminal to view BLE connection logs, status messages, and monitor data transmission between the two devices during Task 5.

Hardware Setup

CC1352P7 LaunchPad: a dual-band wireless MCU with an ARM Cortex-M4F processor, Sub-1 GHz and Bluetooth 5.2 wireless support, chosen for its low power and dual-band operation.

Kit 1 (Peripheral): ran the simple_peripheral firmware, advertised BLE services, responded to requests, exposed the LED as a GPIO controlled via BLE writes, and sent notifications when its

button was pressed.

Kit 2 (Central): ran the simple_central firmware, scanned for and connected to Kit 1 using its MAC address, and sent messages to Kit 1 via BLE.

BLE Communication Flow

1. Peripheral advertising: Kit 1 broadcasts BLE advertisements containing its device name, service UUIDs, and MAC address
2. Central scanning: Kit 2 scans for nearby BLE devices and matches Kit 1 by MAC address or name
3. Connection establishment: Kit 2 initiates a connection request, which Kit 1 accepts
4. Service discovery: Kit 2 discovers Kit 1's GATT services and characteristic handles
5. Mobile communication: the phone can also connect to Kit 1 as a client, alongside the kit-to-kit BLE connection

Key Learnings

BLE concepts:

- GATT (Generic Attribute Profile) defines how data is exchanged, organized into services (collections of characteristics) and characteristics (individual data values), each identified by a UUID
- Advertising and scanning are how devices announce themselves and discover each other
- Central/Peripheral describes the connection roles, while Client/Server describes the data exchange roles

TI SimpleLink SDK:

- TI-RTOS provides the real-time operating system underlying the BLE stack
- Board files provide hardware abstraction for specific boards
- The SDK is organized into a clear project structure with debug, release, and flash build configurations

Embedded development:

- Code Composer Studio (CCS) was used as the IDE
- Debugging used JTAG/SWD via the XDS110 debugger
- UART with Putty was used for serial debugging
- Flashing was used to load firmware onto the MCU

Tools & Technologies

- IDE: Code Composer Studio (TI)
- SDK: SimpleLink CC13xx/CC26xx SDK

- Hardware: TI CC1352P7 LaunchPad (2 kits)
- Protocol: Bluetooth Low Energy (BLE) 5.2
- App: nRF Connect (mobile debugging)
- Debug: Putty (serial terminal)
- MCU architecture: ARM Cortex-M4F

Outcome

- Transitioned from Raspberry Pi to TI CC1352P7 embedded development and set up the full toolchain (CCS + SimpleLink SDK)
- Verified basic GPIO functionality with a button-controlled LED example
- Established BLE communication between a mobile phone and a microcontroller using nRF Connect
- Extended BLE communication to a direct connection between two microcontrollers (peripheral and central roles)
- Learned to debug BLE connections and data exchange using UART and Putty